Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Veri saklama (data storage) ve bit
  - Bilgisayardaki tüm bilgiler 0 ve 1 ile kodlanır. En küçük birim bittir (binary digit).
  - 0/1 kullanımının temelinde elektrik sinyalinin var ya da yok olması yatar: devreye örneğin 5 voltluk gerilim verildiğinde 1, gerilim olmadığında 0 olarak kodlanır.
  - İkinci bir yorum mantıktır: 1 doğru (true), 0 yanlış (false) olarak düşünülebilir. Bu, Boole işlemlerinin (Boolean operations) temelidir.
  - 1 byte = 8 bit. Bellekte en küçük adreslenebilir birim byte'tır, bit değildir.
  - Birimler: 1 KB = 2^10 byte = 1024 byte, 1 MB = 2^20 byte, 1 GB = 2^30 byte, 1 TB = 2^40 byte. Ardından petabyte, exabyte, zettabyte gelir. Hoca bunların ötesinde brontobyte ve geopbyte adlarını da anar.
  - 8 bit ile 2^8 = 256 farklı değer saklanabilir. Sayılar yalnızca pozitif kabul edilirse bu aralık 0-255'tir.
- Mantıksal kapılar (logical gates) ve Boole işlemleri
  - Mantıksal kapılar elektronik devrelerin işlem yapabilen en küçük birimleri olarak düşünülebilir. Kapılar girdi alır, çıktı üretir. Toplama, çıkarma, veri saklama ve karşılaştırma gibi işler kapıların birleşmesiyle oluşan devrelerle yapılır.
  - Matematikteki mantık işlemlerinin elektroniğe aktarılmış hâlidir. Transistörün ortaya çıkışıyla bu işlemler çok küçük alanlarda ve çok hızlı yapılabilir hâle gelmiştir (eskiden çok yer kaplayan vakum tüpleri kullanılıyordu).
  - Temel dört kapı: AND, OR, XOR ve NOT. Ayrıntıları Lojik Devreler dersinde görülür.
  - AND (2 girdi, 1 çıktı): yalnızca iki girdi de 1 ise 1 üretir.

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

  - OR (inclusive or): girdilerden en az biri 1 ise 1 üretir. Algoritmalarda "iki koşul da sağlanmalı" ise VE, "birinin sağlanması yeterli" ise VEYA kullanılır. Hoca, OR'u bir tür toplama, AND'i bir tür çarpma gibi düşünmenin mümkün olduğunu söyler.
  - XOR (exclusive or): girdiler farklıysa 1, aynıysa 0 üretir.
  - NOT (1 girdi, 1 çıktı): girdinin tersini üretir: 0 → 1, 1 → 0.
  - Bit düzeyinde (bitwise) işlemde iki sayının karşılıklı bitleri tek tek işleme girer. Örnek: 0 (00000000) ile 255 (11111111) bitwise OR işlemine girerse sonuç 255, AND işlemine girerse sonuç 0 olur.
- Sayı sistemleri
  - Konumsal olmayan (non-positional) sistemler: Roma rakamları (I, V, X, C, M). Basamak mantığı yoktur, bu yüzden işlem yapmak zordur.
  - Konumsal (positional, tabana dayalı) sistemler: ikilik (binary, taban 2), sekizlik (octal, taban 8), onluk (decimal, taban 10) ve onaltılık (hexadecimal, taban 16). Bilgisayar dünyasında en çok ikilik ve onaltılık kullanılır.
  - Onaltılık sistemde 9'dan sonra harfler gelir: A = 10, B = 11, C = 12, D = 13, E = 14, F = 15. 16 sayısı onaltılıkta 10 olarak yazılır.

  - Sekizlik sistemde rakamlar 0-7'dir. Onluktaki 8 sekizlikte 10, onluktaki 16 sekizlikte 20 olarak yazılır.
  - Basamak değerleri tabanın kuvvetleridir: onlukta 1, 10, 100; sekizlikte 1, 8, 64; onaltılıkta 1, 16, 256; ikilikte 1, 2, 4, 8, 16… Genel olarak d tabanında basamaklar sağdan sola d^0, d^1, d^2, d^3…
  - Gösterim: sayının tabanı kapanan parantezin sağ altına yazılır, örneğin $(227)_{10}$. Onaltılık sayılar ayrıca `0x` önekiyle de yazılır (örn. `0xE3`). Buradaki x çarpı değildir, onaltılık sistemi gösterir.
  - Tabanın belirtilmesi zorunludur. Örneğin "1000" ikilikte 8, onaltılıkta 16^3 = 4096 eder.
  - İkilik sayılar genellikle 8'in katları kadar bitle yazılır, başa gereken sayıda 0 eklenir. Onluk 0-15 aralığı 4 bitle yazılabilir. 16'yı yazmak için 5 bit gerekir.
  - Hoca 0'ı O harfinden ayırmak için içine çizgi çekerek (Ø) yazar.
- En anlamlı ve en anlamsız bit
  - En sağdaki bit en düşük anlamlı bittir (least significant bit, low-order end). Değişmesi sayıyı yalnızca 1 kadar etkiler.
  - En soldaki bit en yüksek anlamlı bittir (most significant bit, high-order end). Değişmesi sayıyı en çok etkiler (8 bitte 2^7 = 128 kadar).
- Taban dönüşümleri (tahtadaki örnekler)
  - İkilikten onluğa: $(11100011)_2 = 128 + 64 + 32 + 2 + 1 = (227)_{10}$.
  - Onluktan onaltılığa: 227'nin içinde 256 yoktur. 16 ise 14 kez vardır ($14 \times 16 = 224$), kalan 3'tür. 14 = E olduğundan $(227)_{10} = (E3)_{16} =$ `0xE3`. Sağlama: $14 \times 16^1 + 3 \times 16^0 = 227$.
  - $(255)_{10} = (11111111)_2 =$ `0xFF`, çünkü $15 \times 16 + 15 = 255$.
  - Onluktan ikiliğe iki yol vardır:
    * 2'nin kuvvetleriyle: $33 = 32 + 1 = 2^5 + 2^0 \rightarrow (100001)_2$.
    * Art arda 2'ye bölerek: kalanlar tersten okunur.
- Negatif sayıların saklanması
  - Bilgisayarda "eksi" işareti ayrıca saklanamaz. Negatif sayılar da 0 ve 1 kombinasyonlarıyla ifade edilir.
  - İki temel yöntem: ikiye göre tümleyen gösterimi (two's complement notation) ve excess gösterimi (excess notation).
- Bire göre tümleyen (one's complement)
  - Tüm bitler ters çevrilir: 0'lar 1, 1'ler 0 olur. Örnek: 01010001 → 10101110 (ekrandan, 01:25:40).
- İkiye göre tümleyen (two's complement)
  - Önce bire göre tümleyen alınır, sonra sonuca 1 eklenir. Örnek: 01010001 → 10101110 + 1 = 10101111 (ekrandan, 01:25:40).
  - Toplama ikilik sistemde eldeli yapılır: 1 + 1 = 10 (0 yaz, 1 elde).
  - İşaretli (signed) sayılarda en anlamlı bit işaret bitidir (sign bit): 0 ise pozitif, 1 ise negatif.
  - Pozitif sayı doğrudan hesaplanır: 01111111 = 127.
  - Negatif sayının değeri bulunurken önce eksi işareti not edilir, sonra sayının ikiye göre tümleyeni alınır ve çıkan değerin başına eksi konur:
    * 11111111 → tümleyen 00000000 + 1 = 00000001 → −1.
    * 10000000 → tümleyen 01111111 + 1 = 10000000 = 128 → −128.
  - −1'i ikilikte yazmak için: önce +1 yazılır (00000001), sonra ikiye göre tümleyeni alınır → 11111111. Sağlaması, sonucun tekrar ikiye göre tümleyeni alınarak yapılır.
  - 8 bitlik aralıklar: işaretsiz 0 ile 255, işaretli −128 ile 127. İşaret biti ayrıldığı için geriye 7 bit kalır. Bu yüzden işaretli 8 bitte 128 ve 255 yazılamaz.
  - İşaretli sistemde 255 yazmak için daha fazla bit gerekir. Bitler 8'in katı olarak kullanıldığından 16 bit kullanılır: 00000000 11111111.
  - Kısa yol: n bitlik bir sayının işaretsiz değeri 2^n'den çıkarılırsa işaretli (negatif) değerin mutlak değeri bulunur. 8 bitte 11111111 işaretsiz 255'tir → 256 − 255 = 1 → −1. 10000000 işaretsiz 128'dir → 256 − 128 = 128 → −128. 16 bit için 2^16 = 65.536 kullanılır. Hoca sınavlarda bu kısa yol yerine adım adım çözüm bekler.
- Neden bire göre tümleyen kullanılmaz?
  - Bire göre tümleyende sıfırın iki ayrı gösterimi vardır: 00000000 (+0) ve 11111111 (−0). Aynı değer iki farklı kombinasyonla ifade edildiği için bir değer kaybedilir.
  - İkiye göre tümleyende 11111111 doğrudan −1'e karşılık gelir ve sıfırın tek gösterimi

vardır. Bu yüzden işaretli sayılarda ikiye göre tümleyen kullanılır.

- Excess gösterimi
  - Kullanılan bit sayısına göre adlandırılır: 3 bitte excess-4, 4 bitte excess-8, 8 bitte excess-128.
  - Yalnızca en soldaki biti 1, diğerleri 0 olan desen sıfırı temsil eder: 3 bitte 100 = 0, 4 bitte 1000 = 0.
  - Hesaplama kuralı: bit deseninin işaretsiz değerinden excess değeri çıkarılır. Örnek (excess-4): 011 = 3 → 3 − 4 = −1. 110 = 6 → 6 − 4 = 2.
  - Excess-4 ile −4 ile 3 arası, excess-8 ile −8 ile 7 arası, excess-128 ile −128 ile 127 arası ifade edilir. Negatif tarafta her zaman bir değer fazla vardır.
  - Excess-8 tablosu (ekrandan, 02:19:00): 1111 = 7, …, 1001 = 1, 1000 = 0, 0111 = −1, 0110 = −2, 0101 = −3, 0100 = −4, 0011 = −5, 0010 = −6, 0001 = −7, 0000 = −8.
  - Hoca bu gösterimin bir IEEE standardı olduğunu ve birçok mimaride kullanıldığını belirtir.
- İkilik kesirli sayılar
  - Noktanın sağındaki basamaklar 2^−1, 2^−2, 2^−3… değerlerini taşır.
  - Örnek: $(1100.110)_2 = 8 + 4 + 1/2 + 1/4 = 12\ 3/4 = (12{,}75)_{10}$ (ekrandan, 02:32:00).
  - Sorun: nokta bilgisayarda doğrudan saklanamaz. Bunun için kayan nokta gösterimi kullanılır.
- Kayan nokta gösterimi (floating point notation)
  - Bit grubu üç parçaya ayrılır: işaret biti (sign bit), üs (exponent) ve mantis (mantissa). 8 bitlik örnekte: 1 bit işaret, 3 bit üs, 4 bit mantis.
  - Üs kısmı excess gösterimiyle yorumlanır (3 bit → excess-4). Üs, noktanın kaç basamak ve hangi yöne kayacağını belirler.
  - Mantisin başına nokta konur. Üs pozitifse nokta sağa, negatifse sola kaydırılır.
  - Örnek (ekrandan, 02:32:00): 0 | 110 | 1011
    - İşaret 0 → pozitif.
    - Üs 110 = 6 → excess-4'te 6 − 4 = +2.
    - Mantis .1011 → nokta 2 basamak sağa kayar → 10.11.
    - $10.11 = 2 + 1/2 + 1/4 = 2\ 3/4 = (2{,}75)_{10}$. İşaret biti 1 olsaydı sonuç −2,75 olurdu.
  - Sayının negatifliği işaret bitiyle, noktanın sağa ya da sola kayması üs kısmındaki excess gösterimiyle belirlenir. Bu iki kavram birbirinden farklıdır.
  - Gerçek sistemlerde 32 ve 64 bitlik gösterimler kullanılır. Bu durumda üs ve mantis kısımları büyür, işaret biti her zaman 1 bittir.
- 32 bit ve 64 bit işlemciler (öğrenci sorusu)
  - 32 bit bir işlemci tek seferde 32 bitlik sayılarla işlem yapar. Bellek tarafında adresleyebildiği alan 2^32 ile sınırlıdır. Bu yüzden eski 32 bit sistemlerde 4 GB'tan fazla bellek doğrudan adreslenemez.
  - 64 bitte iki ayrı 32 bitlik işlemle yapılacak iş tek bir 64 bitlik işlemle yapılabilir. Bu da büyük sayılarla çalışırken önemli hız avantajı sağlar.

Hocanın Özellikle Vurguladığı Kısımlar

- Tabanı mutlaka yazın
  - Sayının hangi tabanda olduğu, kapanan parantezin sağ altına yazılmalıdır. Aksi hâlde aynı rakam dizisi farklı sayılara karşılık gelebilir.
- İşaretli sayılarla çalışırken bit sayısını 8'in katına tamamlayın
  - 8, 16, 32 gibi değerler kullanılır. 9 bit gibi ara değerlerle çalışılmaz.
- Hesaplamaları adım adım yapın
  - $(11100011)_2$ dönüşümünde basamak kaydırma hatası (2^1 yerine 2^2 almak) sık yapılan bir hatadır. Sınav ve quizlerde basamak değerlerini tek tek yazarak hesaplayın.
- İkiye göre tümleyen kuralı ezberlenmeli
  - Negatif sayının değeri: işaret bitine bak, ikiye göre tümleyeni al, başına eksi koy. Negatif sayıyı yazmak: pozitifini yaz, ikiye göre tümleyenini al. Sağlama: sonucun tekrar ikiye göre tümleyenini al.
- İşaretli sistemde pozitif aralık daralır
  - Bir bit işaret için ayrıldığından, 8 bit işaretli sistemde en büyük sayı 127'dir.
- C dilinde işaretli/işaretsiz
  - C'de bir değişken varsayılan olarak işaretlidir. İşaretsiz istenirse başına `unsigned` yazılır. Bazı dillerde işaretsiz tip yoktur.

Kısa Tekrar Notları

- Bit = binary digit. 1 byte = 8 bit. 8 bit → 256 değer.
- AND: ikisi de 1 → 1. OR: en az biri 1 → 1. XOR: farklıysa 1. NOT: tersini alır.
- Tabanlar: 2 (binary), 8 (octal), 10 (decimal), 16 (hex: A=10 … F=15).
- Basamak değeri = taban^konum. Hex: 1, 16, 256. Octal: 1, 8, 64.
- $(227)_{10} = (11100011)_2 = 0xE3$. $(255)_{10} = 0xFF = 11111111$.
- LSB = en sağdaki bit, MSB = en soldaki bit.
- Bire göre tümleyen: bitleri ters çevir. İkiye göre tümleyen: bire göre tümleyen + 1.
- İşaretli 8 bit: −128 ile 127. İşaretsiz 8 bit: 0 ile 255.
- 11111111 → işaretli −1, işaretsiz 255. 10000000 → işaretli −128, işaretsiz 128.
- Bire göre tümleyenin sorunu: iki sıfır vardır (+0 ve −0).
- Excess-k: değer = işaretsiz değer − k. 100…0 = 0.
- Kayan nokta (8 bit): işaret (1) | üs (3, excess-4) | mantis (4). 0|110|1011 = +2,75.

Detaylı Açıklamalar

Bu derste bilginin bilgisayarda hangi biçimde saklandığı ve bu bilgi üzerinde temel işlemlerin nasıl yapıldığı ele alınır. Tüm bilgi 0 ve 1'lerle kodlanır. Bunun fiziksel karşılığı elektriğin var ya da yok olmasıdır, mantıksal karşılığı ise doğru ve yanlış değerleridir. Bit en küçük birimdir, 8 bit bir byte eder ve bellekte adreslenebilen en küçük birim byte'tır. n bit ile 2^n farklı değer ifade edilebilir. 8 bit için bu 256 değerdir ve sayılar yalnızca pozitif kabul edilirse 0-255 aralığına karşılık gelir.

Bu 0 ve 1'ler üzerinde işlem yapmayı sağlayan yapılar mantıksal kapılardır. AND kapısı iki girdi alır ve yalnızca ikisi de 1 olduğunda 1 üretir. OR kapısı girdilerden en az biri 1 olduğunda 1 üretir. XOR kapısı girdiler farklı olduğunda 1, aynı olduğunda 0 üretir. NOT kapısı tek girdi alır ve tersini üretir. Bu dört temel kapının birleşimiyle toplama, çıkarma, karşılaştırma ve veri saklama gibi işleri yapan devreler kurulur. Bit düzeyindeki işlemlerde iki sayının karşılıklı bitleri tek tek bu kapılardan geçirilir. Örneğin 0 ile 255 bitwise OR işlemine girerse her bit 1 olur ve sonuç 255'tir. AND işleminde ise her bit 0 olur ve sonuç 0'dır. Algoritmalarda da aynı mantık kullanılır: iki koşulun birlikte sağlanması gerekiyorsa VE, birinin yeterli olduğu durumda VEYA kullanılır.

Sayı sistemleri konumsal ve konumsal olmayan olarak ikiye ayrılır. Roma rakamları konumsal değildir ve işlem yapmaya uygun değildir. Bilgisayarda konumsal sistemler kullanılır: ikilik, sekizlik, onluk ve onaltılık. Konumsal bir sistemde her basamağın değeri tabanın bir kuvvetidir: d tabanındaki bir sayıda basamaklar sağdan sola d^0, d^1, d^2 değerlerini taşır. Onaltılık sistemde tek basamakta 15'e kadar değer yazılabilmesi için 10-15 arası A-F harfleriyle gösterilir. 16 sayısı onaltılıkta "10" olarak yazılır, çünkü ikinci basamak 16'lar basamağıdır. Aynı şekilde sekizlik sistemde rakamlar 0-7'dir ve onluktaki 8 sayısı sekizlikte "10" olur. Bir sayının tabanı parantezin sağ altına yazılarak ya da onaltılık için `0x` önekiyle belirtilmelidir. Aksi hâlde "1000" gibi bir dizinin hangi sayıyı temsil ettiği anlaşılamaz.

Taban dönüşümleri tahtada örneklerle gösterilir. $(11100011)_2$ sayısı 128 + 64 + 32 + 2 + 1 = 227 eder. Bunu onaltılığa çevirmek için 227'nin içinde kaç tane 16 olduğuna bakılır: 14 tane (224), kalan 3. 14 onaltılıkta E olduğu için sonuç E3'tür. Onluktan ikiliğe geçerken sayı 2'nin kuvvetlerine ayrılabilir (33 = 32 + 1 = 100001) ya da art arda 2'ye bölünüp kalanlar tersten okunabilir. En sağdaki bit en düşük anlamlı bit (LSB), en soldaki bit en yüksek anlamlı bittir (MSB). Bu adlandırma, o bitteki değişikliğin sayıyı ne kadar etkilediğine dayanır.

Negatif sayılar bilgisayarda ayrı bir eksi işaretiyle saklanamaz. Bu yüzden belirli gösterim kuralları kullanılır. İkiye göre tümleyen gösteriminde en anlamlı bit işaret bitidir. Pozitif bir sayının değeri doğrudan okunur. Negatif bir sayının değerini bulmak için ikiye göre tümleyen alınır: önce tüm bitler ters çevrilir (bire göre tümleyen), sonra 1 eklenir. Çıkan değer sayının mutlak değeridir ve başına eksi konur. Örneğin 11111111'in ikiye göre tümleyeni 00000001'dir, dolayısıyla bu desen işaretli sistemde −1'dir. İşaretsiz sistemde ise aynı desen 255'tir. 10000000 işaretli sistemde −128'dir. Bir sayının negatifini yazmak için önce pozitifi yazılır, sonra ikiye göre tümleyeni alınır. Sağlama için sonucun tekrar ikiye göre tümleyeni alınır ve başlangıçtaki sayıya dönülüp dönülmediğine bakılır. Bir bit işaret için ayrıldığından 8 bitlik işaretli sistemde aralık −128 ile 127'dir. Daha büyük bir sayı gerekiyorsa bit sayısı 8'in bir sonraki katına (16) çıkarılır. Pratik bir kısa yol da vardır: bit desenin işaretsiz de-

ğeri 2^n'den çıkarıldığında negatif değerin mutlak değeri bulunur (8 bit için 256, 16 bit için 65.536). Ancak sınavlarda adım adım yöntem beklenir.

Bire göre tümleyenin negatif sayılar için tercih edilmemesinin nedeni, sıfırın iki ayrı gösterimi olmasıdır: 00000000 artı sıfır, 11111111 ise eksi sıfır gibi davranır. Aynı değeri iki farklı desenle göstermek bir değerin boşa harcanması demektir. İkiye göre tümleyende bu sorun yoktur. 11111111 doğrudan −1'dir.

Negatif sayıları göstermenin diğer yolu excess gösterimidir. Kullanılan bit sayısına göre bir "excess" değeri belirlenir: 3 bitte 4, 4 bitte 8, 8 bitte 128. Yalnızca en soldaki biti 1 olan desen sıfırı temsil eder. Herhangi bir desenin değeri, işaretsiz değerinden excess değerinin çıkarılmasıyla bulunur. Örneğin excess-4'te 011 = 3 − 4 = −1 ve 110 = 6 − 4 = 2'dir. Excess-8 ile −8 ile 7 arası, excess-128 ile −128 ile 127 arası ifade edilir. Her durumda negatif tarafta bir değer fazladır.

Son olarak kesirli sayılar ele alınır. İkilik bir kesirde noktanın sağındaki basamaklar 2^−1, 2^−2, 2^−3 değerlerini taşır. Örneğin 1100.110 sayısı 12 + 1/2 + 1/4 = 12,75'tir. Ancak bilgisayar noktayı doğrudan saklayamaz. Bu yüzden kayan nokta gösterimi kullanılır. 8 bitlik örnekte bitler bir işaret biti, 3 bitlik üs ve 4 bitlik mantis olarak ayrılır. Mantisin başına nokta konur. Üs kısmı excess-4 gösterimiyle yorumlanır ve noktanın kaç basamak kayacağını belirler: pozitifse sağa, negatifse sola. Örnekte 0|110|1011 deseni şöyle çözülür: işaret 0 olduğu için sayı pozitiftir. Üs 110 = 6 − 4 = +2 olduğundan .1011 mantisinde nokta iki basamak sağa kayar ve 10.11 elde edilir. Bu da 2 + 1/2 + 1/4 = 2,75'tir. Sayının işareti işaret bitiyle, noktanın konumu ise üsteki excess gösterimiyle belirlenir. Bu yapı sayesinde hem büyük tam sayılar hem de hassas kesirli sayılar aynı sınırlı bit alanında ifade edilebilir. Gerçek sistemlerde 32 ve 64 bitlik kayan nokta biçimleri kullanılır. Bir sonraki derste yuvarlama/kesme hatası (truncation error) konusuyla devam edilecektir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.